TUTORIAL DE GIT

Capítulo 1. Introducción al control de versiones

Institución: UNIAGRARIA
Asignatura: INTRODUCCIÓN A AL CIENCIA DE DATOS
Estudiante: CAROL DAYANA ORTIZ
Docente: CARLOS ANDRÉS GÓMEZ VASCO
Fecha: 14/10/2026

1. Introducción

En la actualidad, el desarrollo de proyectos académicos y tecnológicos requiere herramientas que permitan organizar archivos, registrar modificaciones y recuperar versiones anteriores de un documento. Cuando se trabaja con múltiples archivos, es importante mantener un historial que facilite la identificación de los cambios realizados y permita evitar la pérdida de información.

Git es un sistema de control de versiones que permite registrar los cambios realizados en los archivos de un proyecto a lo largo del tiempo. Gracias a esta herramienta, es posible consultar versiones anteriores, comparar modificaciones y mantener un historial organizado del trabajo realizado.

Este tutorial presenta los conceptos fundamentales de Git, la evolución de los sistemas de control de versiones, el funcionamiento básico de la herramienta, su instalación y la configuración inicial necesaria para comenzar a utilizarla.

Además, se relacionan estos conceptos con GitHub, una plataforma que permite alojar repositorios Git en internet y compartir proyectos. Estos conocimientos se aplicarán en la elaboración de documentos de texto y en el desarrollo de la actividad académica propuesta.

2. Objetivos

2.1. Objetivo general

Comprender los fundamentos del control de versiones y el funcionamiento básico de Git para administrar archivos, registrar cambios y mantener un historial organizado de un proyecto.

2.2. Objetivos específicos

- Identificar qué es un sistema de control de versiones y cuáles son sus principales beneficios.

- Reconocer las diferencias entre los sistemas locales, centralizados y distribuidos.
- Comprender los conceptos fundamentales relacionados con el funcionamiento de Git.
- Identificar los requisitos para instalar y utilizar Git en un computador.
- Configurar el nombre y el correo electrónico del usuario.
- Reconocer la utilidad de la línea de comandos para administrar repositorios.

3. ¿Qué es el control de versiones?

El control de versiones es un mecanismo que permite registrar y administrar los cambios realizados en uno o varios archivos durante un periodo determinado.

Su propósito es conservar un historial de las modificaciones para que el usuario pueda consultar versiones anteriores, comparar contenidos y comprender cómo ha evolucionado un proyecto.

Por ejemplo, una estudiante puede elaborar un documento académico y modificarlo varias veces. Sin un sistema de control de versiones, tendría que guardar copias independientes con nombres como documento final, documento_final2 o documento definitivo. Esta práctica puede generar confusión y dificultar la identificación de la versión correcta.

Con un sistema de control de versiones, los cambios pueden registrarse de manera ordenada, acompañados de mensajes que expliquen qué se modificó y por qué.

3.1. Principales beneficios

Entre las ventajas del control de versiones se encuentran:

- **Historial de cambios:** permite consultar las modificaciones registradas.
- **Recuperación de versiones:** facilita volver a estados anteriores del proyecto.
- **Comparación de archivos:** ayuda a identificar diferencias entre versiones.
- **Organización:** permite mantener un registro estructurado del trabajo.
- **Trazabilidad:** facilita identificar qué cambios se realizaron y cuándo se registraron.
- **Colaboración:** permite que varias personas trabajen sobre un mismo proyecto mediante flujos de trabajo organizados.

Estas ventajas no eliminan la necesidad de realizar copias de seguridad ni garantizan que todos los archivos puedan recuperarse si nunca fueron registrados o guardados correctamente.

4. Tipos de sistemas de control de versiones

Los sistemas de control de versiones han evolucionado para responder a diferentes necesidades de almacenamiento, seguimiento y colaboración.

4.1. Sistemas de control de versiones locales

Los sistemas locales almacenan la información del historial en el mismo computador donde se trabaja.

Su principal ventaja es que permiten registrar modificaciones sin depender de un servidor remoto. Sin embargo, si el equipo sufre una falla y no existen copias de seguridad, la información puede perderse.

4.2. Sistemas de control de versiones centralizados

Los sistemas centralizados utilizan un servidor principal que almacena los archivos versionados y el historial del proyecto. Los usuarios acceden a ese servidor para obtener información y registrar cambios.

Este modelo facilita la administración de permisos y la coordinación del trabajo. No obstante, una falla del servidor puede afectar el acceso al historial y dificultar el trabajo de los colaboradores.

Entre los ejemplos históricos de este modelo se encuentran CVS y Subversion.

4.3. Sistemas de control de versiones distribuidos

En los sistemas distribuidos, cada clon completo del repositorio contiene su propio historial. Esto permite realizar muchas operaciones localmente, incluso cuando no hay conexión a internet.

Git es un sistema de control de versiones distribuido. Los usuarios pueden registrar cambios en su computador y posteriormente intercambiar información con repositorios remotos.

Esta característica resulta especialmente útil para proyectos académicos, ya que permite mantener un historial local y publicar los cambios en una plataforma como GitHub.

5. Breve historia de Git

Git surgió en 2005, en el contexto del desarrollo del núcleo del sistema operativo Linux.

La comunidad que trabajaba en este proyecto necesitaba una herramienta que permitiera administrar grandes cantidades de cambios de manera eficiente. Tras un cambio en las condiciones de uso de BitKeeper, herramienta que se utilizaba en ese momento, se impulsó el desarrollo de una alternativa propia.

Linus Torvalds inició el desarrollo de Git con el propósito de contar con un sistema rápido, distribuido, eficiente y capaz de manejar proyectos de gran tamaño.

Desde entonces, Git ha evolucionado hasta convertirse en una herramienta ampliamente utilizada para el desarrollo de software y para el seguimiento de cambios en distintos tipos de archivos.

6. Fundamentos de Git

Para comprender el funcionamiento de Git, es necesario conocer algunas de sus características principales.

6.1. Repositorio

Un repositorio es el espacio en el que Git administra los archivos del proyecto y almacena su historial de cambios.

Puede existir localmente en el computador o en un servidor remoto. El repositorio permite conservar las confirmaciones realizadas y consultar cómo ha evolucionado el proyecto.

6.2. Confirmaciones o commits

Un *commit* es un registro de los cambios preparados que se incorporan al historial del repositorio.

Cada confirmación incluye información que permite identificarla, como su autor, fecha y mensaje descriptivo. Los commits sirven para documentar avances y consultar estados anteriores del proyecto.

Por ejemplo, un mensaje claro sería:

Agrega introducción al tutorial de Git

Este mensaje explica el cambio realizado de manera más precisa que uno como cambios o actualización.

6.3. Git trabaja con instantáneas

Git organiza las confirmaciones como instantáneas del estado de los archivos del proyecto. Cuando un archivo no ha cambiado, Git puede reutilizar la información almacenada en lugar de duplicarla innecesariamente.

Este enfoque permite mantener un historial eficiente y recuperar los estados que se han registrado.

6.4. Operaciones locales

Muchas operaciones de Git se ejecutan directamente en el computador. Por ejemplo, consultar el historial o revisar el estado de los archivos no requiere necesariamente una conexión a internet.

La conexión se necesita para intercambiar información con un repositorio remoto, como sucede al enviar cambios mediante git push.

6.5. Integridad de la información

Git utiliza identificadores basados en funciones hash para identificar los objetos almacenados en el repositorio. Estos mecanismos permiten detectar cambios inesperados en los contenidos.

En versiones actuales, Git utiliza principalmente SHA-1 para identificar objetos y también dispone de compatibilidad con SHA-256 en determinados formatos de repositorio.

6.6. Los tres estados principales

Git distingue tres estados fundamentales para comprender cómo se administran los archivos:

Modificado (*modified*): el archivo ha cambiado en el directorio de trabajo, pero esos cambios todavía no se han preparado para la siguiente confirmación.

Preparado (*staged*): los cambios se han seleccionado para formar parte del próximo commit.

Confirmado (*committed*): los cambios preparados ya se han guardado en el historial local del repositorio.

El flujo habitual de trabajo consiste en modificar archivos, preparar los cambios y confirmarlos. Posteriormente, si se desea compartir el resultado, se envían los commits al repositorio remoto.

7. Diferencia entre Git y GitHub

Aunque sus nombres están relacionados, Git y GitHub cumplen funciones diferentes.

Git es el sistema de control de versiones que se instala en el computador y que permite registrar, comparar y administrar cambios.

GitHub es una plataforma que permite alojar repositorios Git en internet y facilita su consulta, colaboración y gestión.

Por ejemplo, en esta actividad se utiliza Git Bash para ejecutar comandos y administrar el repositorio local. GitHub permite consultar los archivos publicados y verificar que los cambios se hayan enviado correctamente.

Es posible utilizar Git sin GitHub, ya que el historial puede mantenerse localmente. De igual forma, GitHub utiliza Git para gestionar el historial de los repositorios que aloja.

8. Instalación de Git

Para comenzar a trabajar con Git es necesario instalarlo en el sistema operativo.

En Windows, una de las opciones es utilizar Git for Windows, que incluye Git Bash, una terminal desde la que pueden ejecutarse comandos de Git y comandos habituales de Unix.

La descarga oficial se encuentra en:

https://git-scm.com/download/win

Después de instalarlo, se puede abrir Git Bash desde el menú de aplicaciones de Windows.

8.1. Comprobar la instalación

Para verificar que Git está instalado, se ejecuta el siguiente comando:

git --version

El comando muestra la versión instalada. En el equipo utilizado para esta actividad, la salida obtenida fue:

git version 2.56.0.windows.2

Este resultado confirma que Git está disponible desde la terminal.

9. Configuración inicial de Git

Antes de registrar cambios, es recomendable configurar el nombre y el correo electrónico que se asociarán a los commits.

9.1. Configurar el nombre

Se utiliza el siguiente comando:

git config --global user.name "Carol Ortiz"

La opción --global establece la configuración para el usuario del computador, por lo que normalmente no es necesario repetirla en cada repositorio.

9.2. Configurar el correo electrónico

Se ejecuta:

git config --global user.email "tu-correo@example.com"

El ejemplo debe reemplazarse por el correo que se desea asociar a los commits.

9.3. Verificar la configuración

Para consultar el nombre configurado:

git config --global user.name

Para consultar el correo:

git config --global user.email

También se puede consultar la configuración general mediante:

git config --list

Es importante utilizar una identidad adecuada para el trabajo académico y evitar publicar información personal innecesaria en los repositorios.

10. Comandos básicos para comenzar

Los siguientes comandos permiten comprobar la instalación, consultar la configuración y comenzar a administrar un proyecto.

Comando	Función
git --version	Muestra la versión instalada de Git.
git config --global user.name "Nombre"	Configura el nombre del autor de los commits.
git config --global user.email "correo"	Configura el correo asociado a los commits.

git config --list	Muestra la configuración de Git.
git status	Informa sobre el estado de los archivos del repositorio.
git add archivo.txt	Prepara un archivo para la próxima confirmación.
git commit -m "Mensaje"	Registra los cambios preparados en el historial local.
git log	Permite consultar el historial de confirmaciones.
git clone URL	Crea una copia local de un repositorio existente.
git push	Envía confirmaciones locales a un repositorio remoto configurado.
git pull	Obtiene e integra cambios del repositorio remoto.

Estos comandos se estudiarán y aplicarán de forma práctica durante los siguientes capítulos del tutorial.

11. Ejercicio práctico

Para aplicar los conceptos iniciales, se propone el siguiente ejercicio.

Objetivo: comprobar la instalación de Git y verificar la identidad configurada en el computador.

Procedimiento:

1. Abrir Git Bash.
2. Ejecutar git --version para comprobar que Git está instalado.
3. Configurar el nombre con git config --global user.name "Carol Ortiz".
4. Configurar el correo electrónico con git config --global user.email "tu-correo@example.com", sustituyendo el ejemplo por el correo elegido.
5. Consultar el nombre con git config --global user.name.
6. Consultar el correo con git config --global user.email.
7. Registrar capturas de pantalla que evidencien los comandos ejecutados y sus resultados.

Resultado esperado: contar con una instalación funcional de Git y una identidad configurada para registrar los cambios de los proyectos.

En la actividad realizada, la instalación fue verificada y el nombre y correo se configuraron correctamente. También se creó un repositorio remoto en GitHub y se estableció una conexión con el repositorio local.

12. Conclusiones

El control de versiones es una herramienta fundamental para mantener organizados los archivos de un proyecto y conservar un historial de las modificaciones realizadas. Su uso facilita la comparación de versiones, la identificación de cambios y la recuperación de estados previamente registrados.

Git se destaca por ser un sistema distribuido que permite trabajar localmente y sincronizar posteriormente la información con repositorios remotos. Para utilizarlo de forma adecuada, es importante comprender los estados de los archivos y la función de las confirmaciones o commits.

Asimismo, se identificó que Git y GitHub son herramientas complementarias, pero diferentes: Git administra el control de versiones, mientras que GitHub proporciona una plataforma para alojar y compartir repositorios.

Finalmente, la instalación y configuración inicial de Git constituyen el primer paso para desarrollar un flujo de trabajo organizado. Estos fundamentos permitirán aplicar comandos de seguimiento, confirmación y publicación de cambios en los documentos de la actividad académica.

13. Referencias bibliográficas

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. Disponible en la documentación oficial de Git, en español:

https://git-scm.com/book/es/v2

Git. (s. f.). *Acerca del control de versiones*. Pro Git, capítulo 1. https://git-scm.com/book/es/Empezando-Acerca-del-control-de-versiones

Git. (s. f.). *Fundamentos de Git*. Pro Git, capítulo 1. https://git-scm.com/book/es/v2/Inicio---Sobre-el-Control-de-Versiones-Fundamentos-de-Git.html

Git. (s. f.). *Configurando Git por primera vez*. Pro Git, capítulo 1. https://git-scm.com/book/es/v2/Inicio---Sobre-el-Control-de-Versiones-Configurando-Git-por-primera-vez.html

Git. (s. f.). *Instalación de Git*. Pro Git, capítulo 1. https://git-scm.com/book/es/v2/Inicio--Sobre-el-Control-de-Versiones-Instalaci%C3%B3n-de-Git.html